AI Agent City — Multi-Client Platform Proposal

Oct 27, 2026 · @Abdul

Executive summary

We propose turning the single Fast Sales virtual office into **AgentCity** (working name): a multi-tenant platform where an agency runs AI-staffed offices for many clients, all shown as one living 3D city. Each client gets a building, each team a floor, and each AI agent a desk. Behind them are real CRM data, real outbound messages and a full audit trail.

The pilot already proves the core loop. Seven agents work a live GoHighLevel account: they answer new leads in under a minute, follow up, report daily, and stay grounded in an 871-chunk course knowledge base. A 3D office shows their work live. What is missing is the layer that makes this repeatable: tenant isolation, self-serve onboarding, a shared agent runtime, billing, and white-label controls.

The ask is a phased build. Phase 1 hardens the pilot into a three-tenant beta. Phase 2 adds the city and the agency console. Phase 3 opens a marketplace of agents and integrations. Each phase ends at a gate tied to paying clients, not features.

What the pilot already proves

The Fast Sales office is live today and exercises every layer the city needs, at a scale of one client.

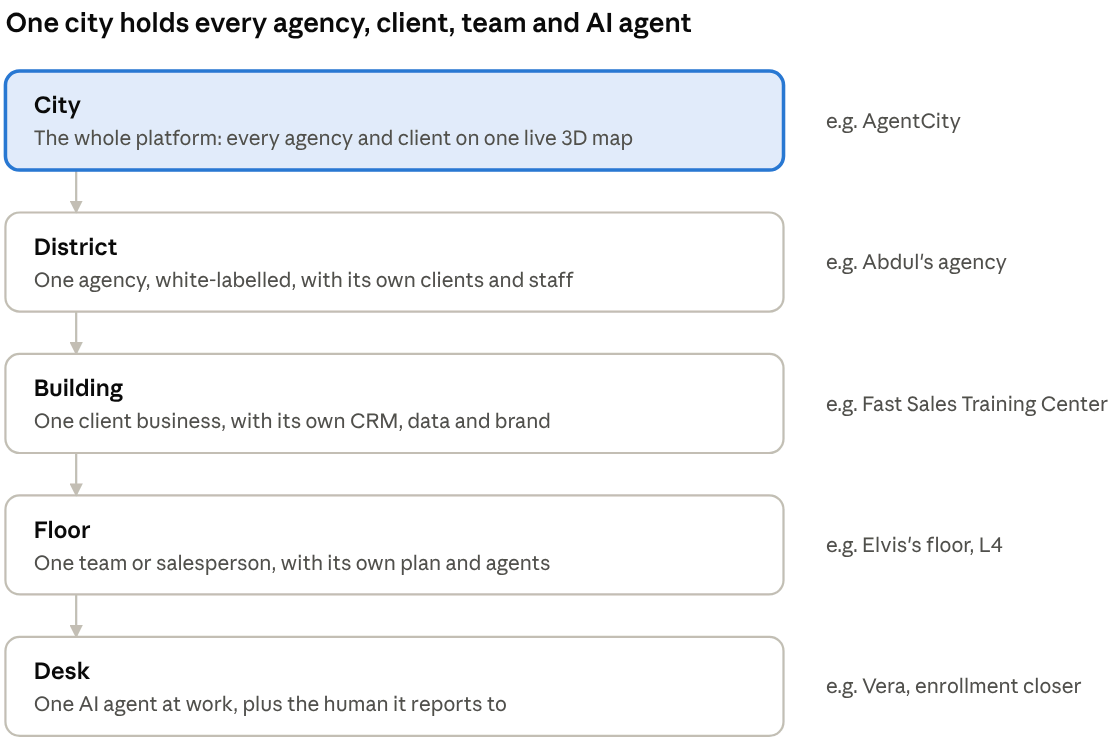
Capability	In the pilot today	What changes for the city
AI agents	7 agents (CEO report, closer, follow-up, content, ads, students, dealers) running as n8n workflows	One shared agent runtime; agents become versioned templates any client can hire
CRM link	GoHighLevel private integration, pipelines and stages matched by name	Connector layer for GHL, HubSpot and Salesforce, with per-tenant credentials in a vault

Capability	In the pilot today	What changes for the city
Context	Lead, pipeline and conversation context, plus an 871-chunk course knowledge base	Per-client knowledge bases with vector search and source citations
Live view	Three.js office with human characters, city outside, day/night, City tab	One shared city: each client a building, zoom from skyline to desk
Access	Roles, 48-hour sign-in links, email codes, audit log	Agency, client and user hierarchy with SSO and passkeys
Delivery	GitHub to Hostinger auto-deploy	Containerised services, staged releases, per-tenant feature flags

Lessons carried over: agents must read context before writing, never double-text a lead, and route anything risky to a human approval queue.

The vision: a living city of AI-run offices

The city is both the product's map and its control room. An agency owner sees every client as a building on the skyline: lit windows show agents at work, and a red beacon flags a building that needs a human. Zooming in moves from district to building to floor to the desk where an agent is texting a lead right now.

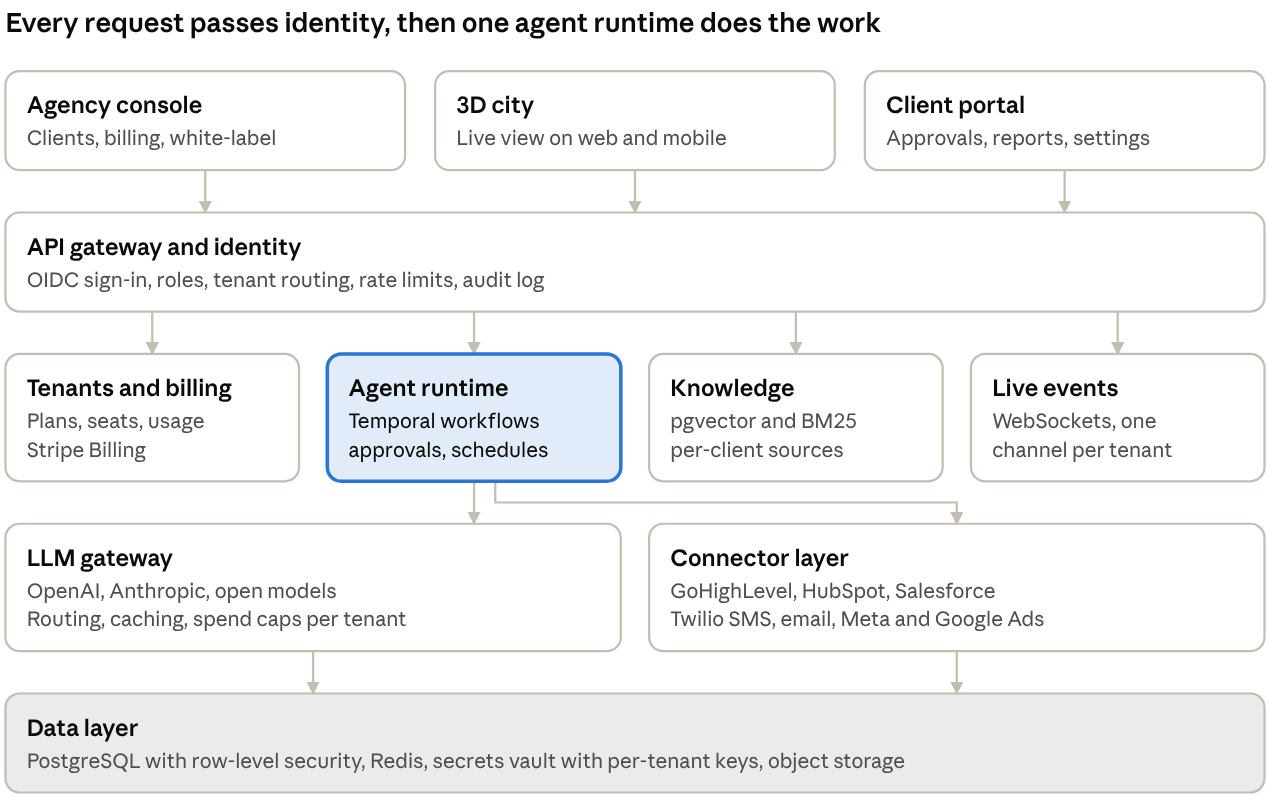


city hierarchy · 5 levels

Each level adds its own permissions, data and billing, so the same picture is also the security model. Busy districts glow, quiet ones dim, and live traffic between buildings shows leads, messages and bookings flowing.

Platform architecture

One shared platform serves every tenant. Isolation comes from identity and the data layer, not from separate copies per client. That keeps upgrades to a single release while each client's data stays sealed.



platform architecture · 5 layers

The agent runtime is the heart of the platform. Each agent run is a durable workflow: it reads CRM and knowledge context, asks the LLM gateway for a draft, checks its guardrails, then sends through a connector or waits for a human approval. Every step emits a live event, and that is what moves the characters in the city.

Technology stack

The stack moves from a PHP-and-JSON pilot to typed, event-driven services. Every choice below is open source or a managed service with an exit path.

Layer	Choice	Why
City front end	Next.js + React, React Three Fiber (Three.js), drei, instanced meshes, LOD and Draco/KTX2 assets	One scene can hold hundreds of buildings at 60 fps on a laptop and stay smooth on phones
Live updates	WebSockets (Socket.IO or Ably) with per-tenant channels	Agents' actions show in the city within a second

Layer	Choice	Why
API	TypeScript, NestJS, REST + GraphQL, OpenAPI contracts	One typed contract shared by the console, the city and partners
Agent runtime	Temporal durable workflows; agents as versioned templates (prompt, tools, guardrails)	Retries, schedules and human approvals survive restarts; no lost follow-ups
AI layer	LLM gateway (OpenAI, Anthropic, open models) with routing, caching, cost caps per tenant	The best model per task, and spend that cannot run away
Knowledge	PostgreSQL + pgvector, hybrid search (BM25 + embeddings), citations	Answers grounded in each client's own material
Data	PostgreSQL with row-level security; Redis for queues and rate limits; S3-compatible storage	Hard tenant isolation inside one database
Integrations	Connector SDK: GoHighLevel, HubSpot, Salesforce, Twilio, SendGrid, Google, Meta Ads; n8n kept for custom client flows	New CRMs plug in without touching agents
Identity	OIDC (Keycloak or Clerk), SSO for agencies, passkeys + email codes	Enterprise sign-in without passwords to manage
Billing	Stripe Billing: seats, agent plans, metered usage	Usage-based pricing from day one
Platform	Docker, Kubernetes (or Fly.io to start), Terraform, GitHub Actions	Repeatable environments and zero-downtime releases
Observability	OpenTelemetry, Grafana, Sentry; an LLM trace per agent run	Every message an agent sends can be explained

Security, data isolation and compliance

No client can ever see, or be messaged by, another client's agents. The design enforces this at four levels.

1. **Tenant isolation.** Every row carries a tenant id, and PostgreSQL row-level security enforces it in the database, not just the code. Each tenant's CRM tokens sit in a secrets vault under their own encryption key.
2. **Access.** Roles cascade: agency owner, agency staff, client admin, client viewer. Sign-in uses SSO or passkeys with email codes. Sensitive changes need step-up confirmation, and sessions are bound to the device.
3. **Agent guardrails.** Agents send only through approved channels and templates. They respect DND and quiet hours, and they cap messages per lead per day. Anything outside policy goes to a human approval queue.
4. **Audit.** Every agent decision, prompt, model, message and approval is logged and can be replayed for review.

Compliance targets:

- US texting rules: A2P 10DLC registration per client, opt-out handling and TCPA consent records.
- Email: CAN-SPAM, plus SPF, DKIM and DMARC per sending domain.
- Privacy: GDPR and CCPA data export and deletion per contact.
- Certification: SOC 2 Type I readiness by the end of Phase 2, Type II after.

Agency model: white-label, onboarding, pricing

The agency resells AgentCity under its own brand and earns recurring revenue per client building. Clients see the agency's logo, domain and colours, never ours.

Onboarding in under one day per client:

1. Pick an industry template: training, auto, real estate, home services or clinics.
2. Connect the CRM with a one-click OAuth or private token.
3. Upload the client's material (websites, PDFs, FAQs) to build their knowledge base.
4. Hire agents from the catalogue and set tone, hours and approval rules.
5. Run a 48-hour shadow mode where agents draft and humans approve, then go live.

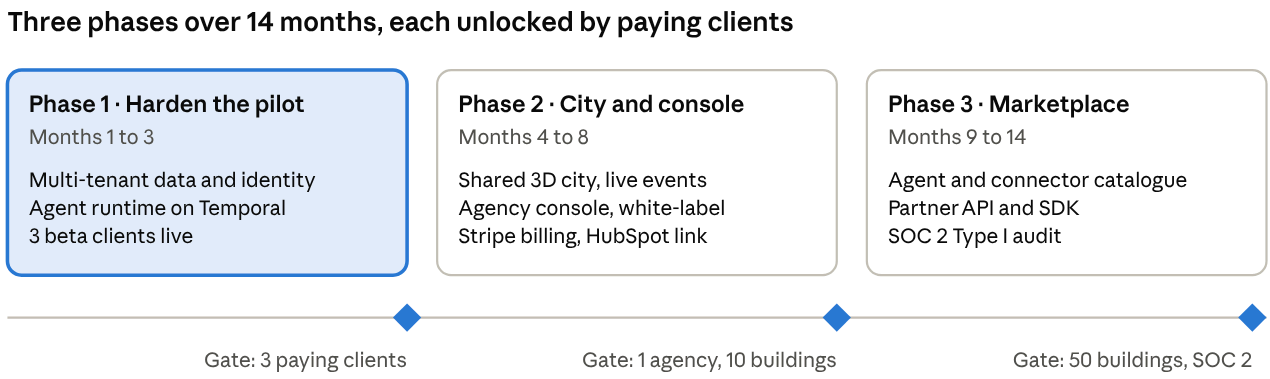
Proposed pricing (to be validated with the first three clients):

Tier	For	Includes	Price (USD / month)
Starter floor	Solo salesperson	2 agents, 1 CRM, 1 user	79
Pro floor	Small team	4 agents, knowledge base, 5 users	199
Full building	Business	7+ agents, approvals, reports, 15 users	399 to 799
Agency district	Agencies	White-label, unlimited buildings, wholesale per building	999 platform + wholesale

AI usage is metered above a monthly allowance, so margins hold on heavy accounts. The Fast Sales floor plans (79, 199, 399) already used in the pilot are the starting point.

Roadmap and phases

The build runs in three phases over about 14 months. A phase starts only when the previous gate is met, so spend follows revenue.



roadmap · 3 phases, 3 gates

Phase 1 reuses the pilot's agents and 3D office, so the first beta clients can go live within weeks rather than months. Timelines are estimates to confirm once the team and budget are set.

Risks, team and next steps

Risk	Mitigation
Texting rules (A2P 10DLC, TCPA) delay a client's go-live	Registration is the first onboarding step; email and in-CRM tasks work while texting is pending
An AI agent sends a wrong or off-brand message	Shadow mode, approval rules, per-lead message caps and grounding in the client's own knowledge base
AI costs grow faster than revenue	Per-tenant spend caps, model routing to cheaper models for routine work, metered overage
CRM API limits or scope changes	Connector layer with backoff, cached reads, and alerts when a scope is missing
The 3D city is slow on older phones	Automatic low-detail mode, 2D dashboard fallback, the city optional per user

Core team for Phases 1 and 2: a tech lead (full-stack, TypeScript), a 3D/front-end engineer, an AI/agent engineer, and a part-time designer and QA. Abdul leads delivery and the client relationship.

Next steps:

- [ ] Confirm the working name and the first three beta clients
- [ ] Approve Phase 1 scope and budget
- [ ] Validate pricing with the beta clients
- [ ] Start A2P 10DLC registration for each beta client